

SkillCorpus: Consolidating and Evaluating the Open Skill Ecosystem for Real-World LLM Agents

Yanze Wang^{1,2,3*}, Pengfei Yao^{1,2*}, Tianyi Sun^{1,2,3}, Chuanrui Hu^{1,2†}, Yan Xiao^{1,2}, Xiaotian Luo^{1,2}, Yunyun Han^{1,2}, Yifan Chen^{1,2}, Jun Sun^{3‡}, Yafeng Deng^{1,2‡}

¹EverMind ²Shanda Group ³Peking University

{yanze.wang, yaopengfei, tianyi.sun, chuanrui.hu, hanyunyun, yifan.chen, dengyafeng}@shanda.com,

{yan.xiao, xiaotian.luo}@evermind.ai, jsun@pku.edu.cn

Abstract

Agent skills, `SKILL.md` files that package reusable procedural knowledge for an LLM agent, are a popular mechanism for extending agent capabilities. Public repositories now host them in large and growing numbers, yet these artifacts are fragmented, redundant, and uneven in quality, and their value in practice is unclear. A core question remains open, namely how to consolidate this open-source `SKILL.md` ecosystem into a single usable corpus, and what bounds its benefit on real-world agent tasks. We present **SkillCorpus**, a framework that aggregates, curates, matches, and evaluates the open skill ecosystem at scale. It filters ~821,000 crawled skills through a multi-stage pipeline into 96,401 skills organised by a 16-class taxonomy and three quality facets (utility, robustness, safety), and pairs them with a fine-tuned retrieval-and-selection stack that matches task-relevant skills. We evaluate end-to-end across three benchmarks (SkillsBench, GDPVal, QwenClawBench), two harnesses, and two open backbones with a frontier robustness check. Integrating SkillCorpus yields consistent gains across all three benchmarks, largest on SkillsBench (+7.5 pp). An operational analysis traces the gains to a coverage boundary and a harness boundary. SkillCorpus is, to our knowledge, the first end-to-end account of when a curated, retrieval-served community corpus improves real agent tasks, and where it does not. The dataset, models, and code will be released upon acceptance.

1 Introduction

Large language models increasingly act as autonomous agents that orchestrate long-horizon executable tasks rather than answer isolated queries (Yao et al. 2023; Shinn et al. 2023; Park et al. 2023). Yet on composite real-world workloads, even the strongest frontier systems remain measurably bounded (Merrill et al. 2026; OpenAI 2025b). This gap is difficult to close by raw model scaling alone. A growing consensus instead treats agent capability as a layered system, in which a frozen foundation model is paired with externalised skills and an agent harness that loads and executes them (Anthropic 2025a). A skill packages reusable procedural knowledge invoked at inference time. Unlike opaque,

slow-to-update model weights or transient in-context examples, it is interpretable and modular. The same externalise-and-retrieve pattern is already well-established for tool use, where agents retrieve and invoke calls over large API inventories (Qin et al. 2024; Patil et al. 2023) and the recent Model Context Protocol (MCP) further standardises access to external tools and resources (Hou et al. 2025; Lin et al. 2025). Skills extend this pattern from external tool calls to procedural knowledge itself.

Industry-grade harnesses now ingest skill files routinely (Anthropic 2025b; OpenAI 2025a), and a community ecosystem has formed around the `SKILL.md` specification (Anthropic 2025a), now spanning thousands of public repositories. Whether this abundance actually improves agent performance on real tasks, and what it takes to make it do so, remains unsettled. Curated skills paired one-to-one with tasks raise pass rates on average but with sharply heterogeneous per-domain effects (Li et al. 2026b), while uncuration community libraries can fail to improve over a no-skill baseline (Li, Tagkopoulos, and Tagkopoulos 2025; Liu et al. 2026b). Two gaps stand in the way. **First**, we know of no released corpus that is at once broadly aggregated, strictly quality-gated, and cleared for open redistribution. Existing artifacts draw from only one or two source channels (Li, Tagkopoulos, and Tagkopoulos 2025; Liu et al. 2026b), and those that filter for quality show no evidence of broad coverage at scale (Liang et al. 2026). **Second**, the downstream value of a community corpus on real agent tasks is largely uncharacterised. Prior evaluations pair tasks with hand-picked oracle skills rather than a deployed corpus (Li et al. 2026b), run in simulated environments (Liang et al. 2026), or measure in-the-wild usage through a single harness over limited sources (Liu et al. 2026b).

To close both gaps, we present **SkillCorpus**, a single end-to-end framework that consolidates the open `SKILL.md` ecosystem into a curated, deployable corpus and evaluates it on real-world agent tasks — making it possible to measure when community skills help real agents, and where they do not.

Our contributions are threefold:

- **Framework.** We unify aggregation, curation, matching, and real-world evaluation of the open `SKILL.md` ecosystem, turning a fragmented, redundant, and uneven pool

*These authors contributed equally.

†Project leader.

‡Corresponding author.

of community skills into a curated foundation for reuse at scale.

- **Open resource.** We release the resulting 96,401-skill corpus (curated from $\sim 821,000$ raw files), its fine-tuned retrieval-and-selection stack, and the full curation, training, and evaluation code, all licence-audited and 100% OSI-permissive — a fully open, ecosystem-scale resource for skill-augmented agent research.
- **Findings.** Evaluating across three real-world agent benchmarks, independent harnesses, and open-to-frontier backbones (§4), we find that integrating SkillCorpus consistently improves agent performance, with gains modulated by corpus coverage and the harness, characterising the real utility and limits of community skills.

2 Related Work

2.1 Skill acquisition and curation

Work on the skill layer divides by when skills enter the system. One strand acquires them at run time, while the agent is solving tasks. Voyager grows a skill library through environment exploration (Wang et al. 2024), Toolformer trains the model to invoke tools self-supervisedly (Schick et al. 2023), ToolLLM and Gorilla scale API calling to large tool inventories (Qin et al. 2024; Patil et al. 2023), and recent agent-evolution work lets agents redesign their own scaffolding (Zhou et al. 2026; Lin et al. 2026). All of these build on agent harness scaffolds (Yao et al. 2023; Shinn et al. 2023). The other strand, which this paper occupies, prepares skills before the agent ever runs. We curate a community-contributed corpus at ingest time, so that runtime systems have a trustworthy library to draw from. This ingest-time curation mirrors data-quality work in language-model training, where deduplication (Lee et al. 2022) and quality-based data selection (Zhou et al. 2023) improve downstream models more than raw scale.

Released community skill corpora vary in emphasis. Liang et al. (2026), concurrent with our work, releases a 200K-skill ontology rated along five quality rubrics and linked by a skill-relation graph. It measures benefit on simulated environments, with skills synthesised from each benchmark’s own expert trajectories rather than on real tasks served by a deployed corpus. Li, Tagkopoulos, and Tagkopoulos (2025) pairs a 36K uncurated library with a four-stage retrieval pipeline and treats quality only implicitly, through artifact-density signals. Ling, Zhong, and Huang (2026) provides a data-driven characterisation of a smaller curated set. None of these releases reports a source-licence audit, so their redistribution rights are left unresolved. Nor does prior work tie these per-skill scores to their downstream operational role. We introduce a compact three-facet framework chosen for measurable facet separation rather than rubric coverage, and characterise its operational signal directly (§4.6).

2.2 Skill retrieval and evaluation

Skill retrieval architectures range from lexical and dense recall to learned skill routers (Zheng et al. 2026), supported by dedicated retrieval benchmarks (Cho, Kang, and Kim 2026), corpus-size scaling analyses (Chen et al. 2026), and

dependency-aware composition (Liu et al. 2026a). A complementary line of work emphasises skill orchestration and context-injection over retrieval per se. Li et al. (2026a) couples capability-tree retrieval with DAG-based orchestration at $\sim 200K$ scale, and Meng, Wang, and Fang (2026) reports +11.7% on SkillsBench via a skill graph plus compact compilation. That gain, however, is measured against the prior state-of-the-art injection method rather than a no-skill or community-corpus baseline, so it is not directly comparable to our end-to-end Δ . The retrieval strand reports ranking metrics on its own curated pools rather than task outcomes. Our stack is fine-tuned on the deduplicated corpus it serves and adds an LLM selector that reads full skill bodies before injection, with both its standalone retrieval performance and its end-to-end effect on task pass rates measured directly.

On evaluation, Li et al. (2026b) characterises oracle-skill efficacy, and Liu et al. (2026b) reports diminishing returns on uncurated libraries at scale. Neither evaluates a single community-curated corpus under a deployable retrieval-and-selection stack across multiple real-world benchmarks. §4 fills this gap with a cross-benchmark, cross-harness, multi-backbone measurement of end-to-end task gains.

3 SkillCorpus

SkillCorpus is a single end-to-end framework that turns the open `SKILL.md` ecosystem into a curated, deployable corpus and measures its effect on real-world agent tasks. As shown in Figure 1, it runs in four stages: aggregation and curation of the ecosystem into a corpus (§3.1–§3.3), matching of task-relevant skills via a fine-tuned retrieval-and-selection stack (§3.4), and end-to-end evaluation on real-world agent tasks (§4).

3.1 Multi-source aggregation and the curation pipeline

The corpus’s value is its coverage, spanning the whole public ecosystem rather than a single channel. The crawl is driven by a single machine-readable source registry of 62 sources, detailed in Appendix A.6. The registry spans five ingestion mechanisms, from directly cloned repositories and awesome-list scrapes to marketplace index APIs, sitemap crawls, and JSON catalogs, and every mechanism feeds one shared discovery entry point. Our crawl gathers $\sim 821,000$ raw `SKILL.md` files. After source selection and repository-level de-duplication, 25,159 GitHub repositories feed the six-stage pipeline, which curates them into the released active set.

Figure 1 visualises the pipeline. Each skill follows the `SKILL.md` convention of Anthropic (2025a) — a Markdown body with optional executable resources; Stages 1–2 apply structural filters on this contract (parse and length/form). Stage 3 deduplicates. Stage 4 runs the LLM judge that emits the 3-facet quality score and 19 safety/quality flags. Stage 5 applies the safety hard-gate plus Open Source Initiative (OSI)-permissive licence filter (Table 5). Stage 6 attaches source-prior shrinkage, the 1024-dim retrieval embedding, and index entries. Parse and deduplication dominate

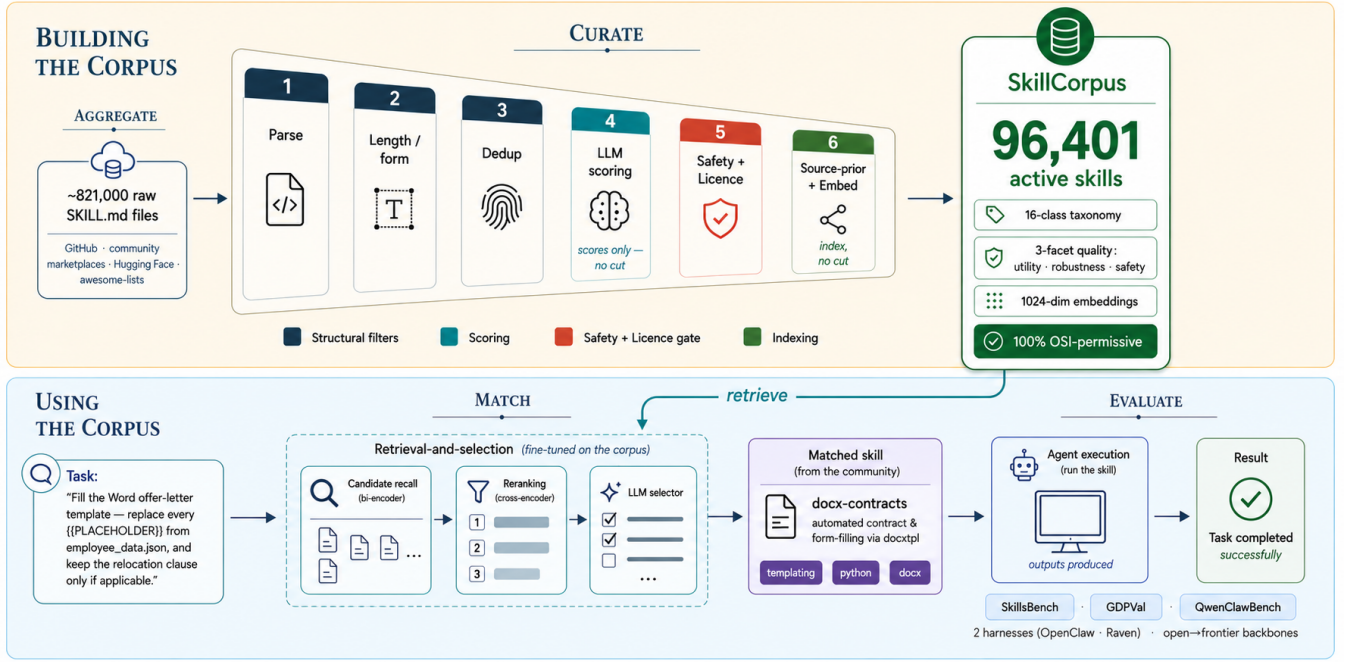


Figure 1: The SkillCorpus framework. Building the corpus (top), we aggregate community `SKILL.md` files from public source channels and curate them through a six-stage funnel into a 96,401-skill corpus organised by a 16-class taxonomy and three quality facets. Using the corpus (bottom), a fine-tuned retrieval-and-selection stack matches skills to an incoming task and an agent runs the selected skill end-to-end. The worked example traces one real task from request to result, and we evaluate this pipeline across three real-world benchmarks, two harnesses, and open-to-frontier backbones.

the funnel, while the remaining filters contribute a few percent.

Two-tier deduplication (stage 3). Deduplication is the funnel’s steepest reduction ($283,844 \rightarrow 101,111$, -64%), in two tiers. An exact tier collapses 169,465 content- and name-fingerprint duplicates, 59.7% of the stage input, which reflects how heavily the ecosystem replicates the same artifacts across repositories. A semantic tier embeds each survivor (1024-dim) and flags pairs above cosine 0.90, and pairs above 0.995 merge automatically. The 66,751 borderline pairs are adjudicated by an LLM judge, which confirms 25.4% of pairs and collapses 13,268 near-duplicate skills. The resulting low-overlap distribution also lets the retrieval stack train on the corpus itself.

3.2 Three-facet quality framework

We decompose skill quality into three facets, each capturing a distinct concern, rather than collapsing them into a single composite or a flat multi-dimensional list. These facets serve curation and retrieval ranking, with the safety facet additionally serving the release gate, as characterised in §4.6.

Three facets. The utility facet scores the description alone, asking whether the task class is well-scoped with clear triggers and applicability conditions. The robustness facet scores the body content and its consistency with the description’s promise. A body that is internally coherent but silently delivers something narrower than promised is penalised here.

The safety facet captures harm and risk along categories such as prompt injection, command injection, credential leakage, and unsafe execution. By design, the utility and robustness facets are scored independently. Description-level issues only affect utility, and body-level issues only affect robustness.

Flag taxonomy. The 19-flag vocabulary is partitioned across the three facets (2 utility-bound, 6 robustness-bound, 11 safety-bound), with the full list in Appendix A.1. Each flag, when fired, constrains its assigned facet’s score. Five flags act as hard gates: `prompt_injection`, `cmd_injection`, `unsafe_exec`, `auth_bypass`, and `csam_risk`. Firing any one of these forces the quality score to zero and excludes the skill from the released active set. The remaining fourteen flags act as soft signals. When such a flag fires, the LLM judge caps the corresponding facet score. A fired `placeholder` flag, for example, holds robustness at ≤ 4 . No additional numeric deduction is applied, so the same observation is not double-counted.

Facet separation. The three facets are not redundant. The safety facet is markedly more independent from the two craftsmanship facets ($r(u, s)=0.15$, $r(r, s)=0.30$) than those are from each other ($r(u, r)=0.59$), matching the design, since u and r both track artifact craftsmanship while s targets an orthogonal harm axis on which a well-crafted skill can still be unsafe. We read these contrasts qualitatively and do not claim construct-validity orthogonality. Rigorous validation would require independent human gold annotation

(Liang et al. 2026). The off-diagonal-mass analysis, the per-pair figure, and the score-bucket distribution are given in Appendix A.2.

Score formula. Following the LLM-as-judge paradigm (Zheng et al. 2023), the judge outputs three integer subscores u, r, s on $\{0, \dots, 10\}$, normalised to $[0, 1]$. These combine into $\text{content_q} = 0.50u + 0.35r + 0.15s$, attenuated by a factor $0.5 + 0.5(s - 0.3)/0.4$ when the safety subscore is marginal ($s \in [0.3, 0.7]$). The final per-skill score, or composite quality score, shrinks content quality toward a per-source prior (Appendix A.3):

$$\text{score} = 0.85 \text{content_q} + 0.15 \text{prior}_{\text{src}} + b,$$

where b is a small structural bonus (+0.05 for skills shipping `scripts/`, +0.02 for `references/`), and the result is clipped to $[0, 1]$.

3.3 Classification, filtering, and release

Taxonomy. We adopt a single-label, 16-class taxonomy designed to cover the observed distribution of community skills while remaining decision-tree classifiable from `name`, `description`, and a short body excerpt. The classifier prompt enforces six conflict-resolution rules, the most consequential being the AI-ML rule that a skill is labelled AI-ML only when its primary deliverable is an AI system (persona, agent, or model), not merely when it uses an LLM internally. The full rule set is released with the classifier prompt.

Distribution. The class distribution on the active release set is summarised in Figure 2. The corpus is software- and data-leaning. `DEV`, `DATA`, `WRITING`, and `DEVOPS-INFRA` together account for over half of active skills. The catch-all `OTHER` class is small (2.01%), indicating that the taxonomy covers the observed distribution well. Every active skill carries exactly one of the sixteen classes.

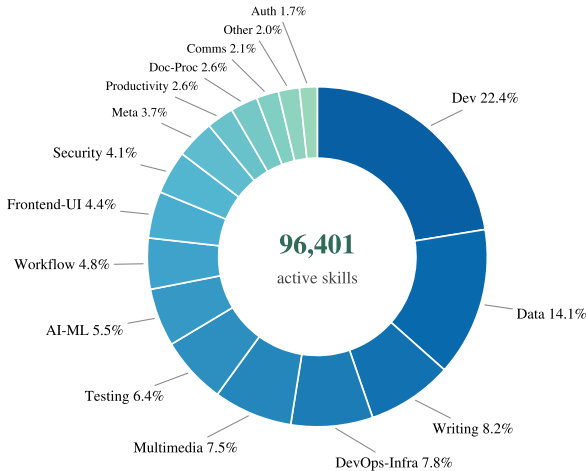


Figure 2: 16-class distribution on the active release set ($N=96,401$, shares in %, sorted clockwise).

Safety and licence filters (stage 5). The two filters run in sequence on the $\sim 101\text{K}$ skills surviving stages 1–4. A safety hard-gate removes 915 skills triggering any of three conditions: a pre-judge `blocked.malware` regex match (Appendix A.4), firing of any of the five LLM-judged hard-gate flags defined in §3.2, or an LLM safety subscore below 3. A licence filter then enriches each source repository’s `spdx_id` via the GitHub API and admits only skills carrying an OSI-approved permissive licence (Table 4), excluding a further 3,795 skills whose source repositories lack a usable permissive licence (per-reason counts in Table 5). Every skill in the resulting 96,401-skill active set is therefore commercially redistributable under its source repository’s declared licence.

Release. The corpus is released as a single artifact comprising the metadata SQLite database, the per-stage ingest report of raw counts at each pipeline stage, the precomputed 3-facet quality cache, the 1024-dim retrieval embeddings, and the retrieval-and-selection stack used in our experiments and released as engineering tooling. All curation source code will be released under a permissive licence upon acceptance.

3.4 Retrieval and selection stack

The corpus is released together with a fine-tuned retrieval-and-selection stack so the artifact ships ready to use rather than requiring a retrieval stack to be built. We use “deployable” in this packaging sense, not as a production-validation claim. The stack follows the standard retrieve-then-rerank pattern of retrieval-augmented generation (Lewis et al. 2020), with three operational stages: candidate recall, rerank, and final selection by an LLM gate.

Recall and rerank. Candidate recall uses an embedding model fine-tuned from Qwen3-Emb-0.6B. A scoring model fine-tuned from Qwen3-Rank-0.6B then reranks the top candidates. Both models are trained on the strictly deduplicated active set. The low-overlap data distribution is intended to give a cleaner contrastive learning signal by preventing functionally identical near-duplicates from being sampled as hard negatives, which could otherwise produce false-negative collisions during contrastive fine-tuning. We adopt this as a training-design rationale rather than an isolated ablation. Recall and rerank operate over a 3,000-character retrieval field distilled from each skill body, avoiding length-induced truncation noise.

LLM selector (gate). Reranked top- k candidates can still include topically irrelevant items, and brief summaries are insufficient for an agent to judge tool fit at execution time. We therefore insert an LLM selector that, given the task query and the full skill body of each reranked candidate, returns 0–2 precise tools for injection into the agent prompt. This trades one additional LLM call for the reliability of full-body inspection while keeping the injected context within strict per-task budgets.

Query rewriter (optional). An optional LLM-based query rewriter normalises domain-specific jargon in the task description to vocabulary that better matches the corpus’s skill

descriptions, and is permitted to short-circuit (no skills required) when the task is clearly outside the corpus’s scope.

We evaluate this stack both as part of the end-to-end pipeline, where it drives the main-grid pass-rate gains, and as a standalone retrieval system on Hit@1 / Recall@10 against published baselines.

4 Experiments and Results

4.1 Setup

Benchmarks, harnesses, and backbones. We evaluate on three third-party agent benchmarks, chosen because frontier LLMs do not saturate them and their domain coverage is complementary. SkillsBench (Li et al. 2026b) contributes 87 deterministic-verifier tasks across 8 domains and is the closest direct analog to our setting. GDPVal (OpenAI 2025b) contributes 220 real-world economic tasks that span legal, financial, design, and writing work. QwenClawBench (Qwen Team, Alibaba Group 2026) adds 100 agent tasks across 8 domains drawn from a real-user task distribution. In total the evaluation covers 407 tasks spanning 26 domain labels (Appendix B.2, Figure 7). We use two open-source SKILL.md-conformant agent harnesses, OpenClaw (Steinberger 2025) and Raven (Raven Contributors 2026), developed independently, paired with two open backbones that span the lightweight-to-large range, Qwen3.5-27B (Q-27B) and Qwen3.5-397B-A17B-GPTQ-Int4 (Q-397B), yielding four (harness, backbone) cells. A frontier-model robustness check on Claude Opus 4.7 verifies that the effect is not specific to the open backbones.

Configurations, inference, and metrics. Each cell is evaluated under a no-skill baseline and under SkillCorpus paired with our retrieval-and-selection stack (§3.4), whose standalone retrieval performance is reported in §4.4. The per-cell Δ therefore measures the end-to-end pipeline effect. All runs use each backbone’s API-default inference configuration. The Qwen models run greedy decoding with their default context and tool-use template, and Opus 4.7 runs at its default thinking effort. Both conditions in a cell share the same setting, so the reported within-cell Δ is unaffected by inference effort and remains valid even though our vendor-default absolute baselines are not directly comparable to published leaderboard numbers obtained under other configurations. For SkillsBench we report the binary pass rate over all 87 tasks, scoring the 4 that do not build in our environment as failures in both conditions. For GDPVal and QwenClawBench we report the mean continuous reward, both on $[0, 1]$ with Δ in percentage points. GDPVal is scored by a GPT-4o judge, from a different model family than our Qwen and Opus backbones, and QwenClawBench by its official grading configuration combining automated checks with an LLM judge (Qwen Team, Alibaba Group 2026). The main grid holds 24 configurations, the four cells evaluated on three benchmarks under both conditions, each averaged over three runs. With the two-condition single-run frontier check, this comes to 74 end-to-end runs. Appendix C.1 reports the ingest and evaluation compute.

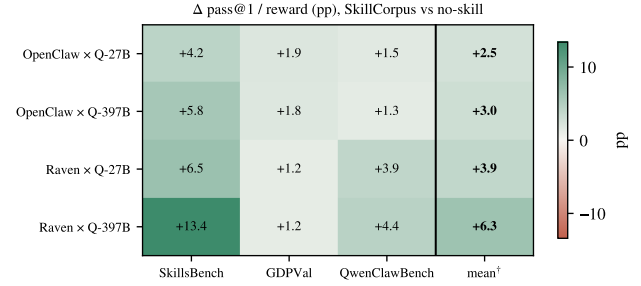


Figure 3: Per-cell, per-benchmark Δ (pp) computed from the absolute values in Table 1. Green = positive, red = negative. [†]Mean over the three benchmarks.

4.2 Main results

Table 1 reports the absolute performance of the no-skill baseline and the SkillCorpus condition (the corpus with our retrieval-and-selection stack) for each of the four (harness, backbone) cells on each benchmark. The per-cell gains (Δ) and their per-cell means are visualised in Figure 3.

On SkillsBench, which has a published oracle baseline (Li et al. 2026b), all four open-backbone cells show positive Δ averaging +7.5 pp over three runs, from +4.2±2.7 on the weakest cell to +13.4±3.1 on the strongest (Raven × Q-397B), with an aligned +8.0 pp on the Claude Opus 4.7 frontier check. In one run the strongest cell rescues 19 baseline-failing tasks while breaking 2 (exact McNemar test, $p < 0.001$). No cell shows a net-negative mean, so at the cell-mean level the corpus behaves as a no-harm attachment whose upside varies by cell, even though individual tasks can still regress (§4.6).

Consistent positive effect across benchmarks. We quantify the effect at the benchmark level by averaging each task’s paired delta across the four cells and computing a task-clustered standard error, $SE = \text{std}/\sqrt{n}$ over the n tasks, so that repeated measurements of the same task across cells are not treated as independent observations. This is the more conservative of two reasonable clusterings. Pooling the cell-task deltas as independent observations would roughly halve the standard errors, and the positive effect holds under either choice. All three benchmarks show a consistent positive effect, each exceeding twice its standard error (bottom row of Table 1). Effect size broadly tracks the headroom a benchmark leaves. SkillsBench starts near a 10% no-skill baseline and gains the most, and within it the low-baseline Industrial & Physical category gains most (+45.5 pp), whereas GDPVal and QwenClawBench start at 65–85% and gain less, where ceiling effects and judge noise both bound the realisable gain. Per-task repetition noise is large on the high-baseline benchmarks, with per-task reward standard deviations of 20–27 pp on GDPVal, so most individual cells are not separately distinguishable from zero. We do not observe the negative tail that Li et al. (2026b) reports for hand-attached skills, nor the failure to improve over a no-skill baseline reported for uncured community libraries (Li, Tagkopoulos, and Tagkopoulos 2025; Liu et al. 2026b), a difference consis-

Cell	SkillsBench no-skill→skill	GDPVal no-skill→skill	QwenClawBench no-skill→skill
OpenClaw × Q-27B	8.8 → 13.0	81.2 → 83.1	65.2 → 66.7
OpenClaw × Q-397B	11.1 → 16.9	82.2 → 84.0	65.7 → 67.0
Raven × Q-27B	10.0 → 16.5	82.6 → 83.8	66.9 → 70.8
Raven × Q-397B	9.2 → 22.6	84.0 → 85.2	68.8 → 73.2
Pooled Δ	+7.5±2.3 ($z=3.2$)	+1.51±0.49 ($z=3.1$)	+2.79±0.70 ($z=4.0$)

Table 1: Main results: absolute performance per cell and benchmark, no-skill baseline → SkillCorpus condition. SkillsBench reports pass@1 averaged over three runs (%), $n=87$ tasks, the 4 that do not build scored as failures). GDPVal reports the mean LLM-judge reward (3-rep) over all 220 tasks. QwenClawBench reports the mean hybrid score (automated checks and an LLM judge), 3-rep mean. Rewards are on $[0, 1]$, shown $\times 100$. Per-cell gains are visualised in Figure 3. The bottom row gives the pooled Δ across the four cells, with a task-clustered standard error over $n=87/220/100$ tasks and two-sided $z=\Delta/\text{SE}$.

tent with the release gates that remove the low-quality mass driving them. Raven outgains OpenClaw on SkillsBench and QwenClawBench across both backbones. On GDPVal neither harness leads, as the gains there are small and individually not significant. The harness thus shapes how much of the gain is realised (§4.6).

4.3 Frontier robustness check

The four main-grid cells span the open capability range (Q-27B, Q-397B). To verify that the pipeline’s effect is not specific to this range, we add a single-cell check on Claude Opus 4.7 on SkillsBench (OpenClaw harness) under the same two-condition protocol. Opus 4.7 reaches a 39.1% no-skill baseline on the 87-task set, and adding SkillCorpus shifts pass@1 to 47.1%, a Δ of +8.0 pp. Although the Opus baseline is several times the open-backbone baselines, its gain is of the same order as the open-backbone cross-cell mean, indicating that stronger frontier models have not made an external curated skill pipeline redundant on this benchmark. We treat this as a robustness check, and a broader frontier grid is left for follow-up work.

4.4 Standalone retrieval

End-to-end Δ entangles corpus quality with retrieval quality. We additionally evaluate the recall-and-rerank stack as a standalone retrieval system on the SkillRouter Easy and Hard tiers (Zheng et al. 2026) and on our SkillCorpus pool, a 75-task core SkillsBench set. Figure 4 reports Hit@1 and Recall@10 across the three pools, with full setup and per-method results in Appendix B.1.

Despite being trained on a different pool, our stack matches SKILLROUTER on the Easy tier and exceeds it on the Hard tier, staying within 1.7 pp on Recall@10. On SkillCorpus it outperforms SKILLROUTER by +10.7 pp Hit@1 and +2.7 pp Recall@10, a margin that partly reflects the matched train and evaluation distribution and the ground-truth skills injected into this pool (Appendix B.1).

4.5 Ablation: corpus and retrieval

To gauge how much of the gain comes from corpus curation versus the fine-tuned retrieval stack, we lesion each in turn on the strongest cell (Raven × Q-397B, SkillsBench). Replacing our retrieval stack with off-the-shelf Qwen3 embedding

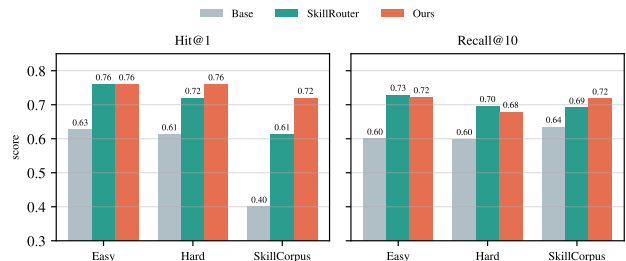


Figure 4: Standalone retrieval performance of BASE, SKILLROUTER, and our fine-tuned stack on the three candidate pools (Hit@1 / Recall@10, exact numbers in Table 7).

and reranking models lowers pass@1 from 22.6% to 13.8%, and replacing the filtered corpus with the $\sim 821,000$ -file raw crawl lowers it to 14.9% (Table 2). Both remain well above the 9.2% no-skill baseline yet well below the full pipeline, so curation and retrieval each contribute and neither alone accounts for the full gain, though the end-to-end Δ also folds in the LLM rewriter/selector layer that this ablation does not isolate. These are single-run numbers with inter-arm gaps of only a few tasks, so we read them as directional and leave a repeated ablation to future work.

Corpus	Retrieval stack	pass@1	Δ
No-skill baseline		9.2	—
Filtered	Fine-tuned (ours)	22.6	+13.4
Filtered	Off-the-shelf Qwen3	13.8	+4.6
Raw crawl	Fine-tuned (ours)	14.9	+5.7

Table 2: Single-run ablation on the strongest cell (Raven × Q-397B, SkillsBench, pass@1 in % over 87 tasks). The full pipeline pairs the filtered corpus with our fine-tuned stack, and each lesion swaps exactly one component. Replacing either the corpus (filtered → raw) or the retrieval stack (ours → off-the-shelf) drops the single-run pass@1 to roughly 14%.

4.6 Discussion: where the pipeline helps, where it does not

The per-skill quality score and 16-class taxonomy play supporting roles rather than driving the gains directly. The taxonomy steers selection benchmark-adaptively, its post-gate class distribution concentrating on each benchmark’s task-relevant classes (Appendix B.3). The composite quality score serves deduplication tie-breaks and retrieval-time ranking rather than per-task prediction, and the active set is gated by safety and licence, not by a score threshold, with only a weak safety-facet signal (Appendix A.2).

When the pipeline helps. Across cells, the per-cell Δ is largest when the benchmark contains procedural sub-tasks beyond the model’s pretraining coverage and the corpus covers the task’s domain. SkillsBench and the QwenClawBench DevOps-INFRA/TESTING/META subset (Figure 7) satisfy both and show the largest cell-level gains.

Why the harness matters. Both harnesses receive identical pre-computed skill selections, and traces show that both ingest them, reading and referencing the injected content. The difference is what happens afterwards. Raven realises a far larger gain than OpenClaw on SkillsBench (+13.4 versus +5.8 pp on Q-397B), although both receive the same skills. The gain also grows with backbone capability, though entangled with the harness. Raven’s +6.5 at Q-27B rises to +13.4 at Q-397B, and the frontier Opus check adds +8.0 pp on OpenClaw, so how much of a skill is realised depends on both the harness and the backbone. What follows is a qualitative case analysis of the two Q-397B cells, not an established mechanism. On the tasks that Raven passes and OpenClaw fails, trace inspection suggests OpenClaw consistently stops after the reasoning phase, writing scripts it never executes and ending without running the verifier, whereas Raven completes an execute–verify–fix loop. This is consistent with skill content converting into verifier passes mainly through such a loop, so that skill utility appears to be a joint property of the corpus and the harness, in line with the harness-utilisation differences Li et al. (2026b) reports across commercial harnesses. On this reading, OpenClaw’s weaker gains on SkillsBench are a harness property rather than a corpus one. The same execution gap carries no penalty on GDPVal, whose LLM-judged writing and document tasks reward output quality rather than a verifier-passing loop, and there the harness gap closes.

Coverage shapes the gain. The gains track how well the corpus covers each task. As a proxy we take the relevance of the best-retrieved skill (its top reranker score), which reflects both whether the corpus holds a matching skill and whether retrieval surfaces it. Binning tasks by this retrieval-match score, mean Δ climbs from +2.2 pp to +25.1 pp from the lowest to the highest bin (Figure 5, pooled over the four cells). Per task the score correlates with Δ at Pearson $r=0.31$ – 0.40 across all four cells (all $|t|>2.9$, $n=83$), and since it is essentially uncorrelated with the no-skill baseline the association survives controlling for task difficulty (partial $r=0.34$ – 0.40 ; Appendix B.2). At the 8-category granularity the gains stay non-negative throughout, with two thin-coverage categories

on the Raven×Q-397B cell at $\Delta\approx 0$ (Mathematics, $N=4$; Finance & Economics, $N=9$). Where the corpus lacks skills for a category, retrieval cannot manufacture them, so the gain floors at zero rather than turning negative. Closing these gaps is a supply problem rather than a retrieval one, calling for generating or self-evolving the skills the crawl does not yet contain rather than better matching over the current corpus. The rare per-task breaks (two on this cell) stem from procedure–task mismatch rather than missing coverage, and Appendix B.4 dissects one (exceltable-in-ppt).

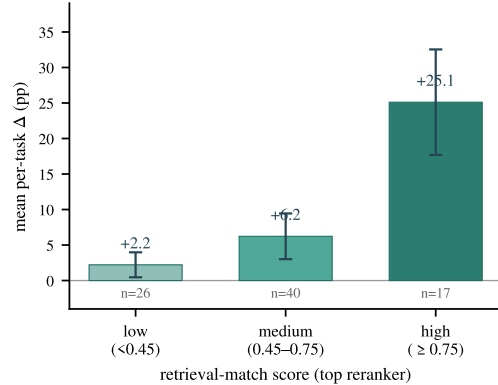


Figure 5: Mean per-task SkillsBench gain by retrieval-match bin (top reranker score of the best-matched skill, a proxy for corpus coverage), pooled over the four cells, with $\pm$ SE error bars. The score is uncorrelated with the no-skill baseline, so the bins are not a headroom artefact. The per-task correlation and raw scatter are in Appendix B.2.

5 Conclusion

SkillCorpus curates $\sim 821,000$ crawled community skill files into a corpus of 96,401 skills, scored along utility, robustness, and safety and matched to tasks by a fine-tuned retrieval-and-selection stack. Across our cross-benchmark, cross-harness, multi-backbone evaluation (§4), integrating the corpus yields consistent positive gains on all three benchmarks. They are large on SkillsBench and modest on GDPVal and QwenClawBench, strongest on the Raven harness, with per-domain effects that remain heterogeneous. Where coverage is thin, growing the corpus by skill generation is the natural next step. The 16-class taxonomy steers benchmark-aware retrieval. The composite quality score serves curation and ranking rather than per-task prediction, and its safety facet is the one component that both gates unsafe skills and carries a suggestive per-task signal. We release SkillCorpus as an open, reusable framework for community skill ecosystems; to our knowledge, it is the first end-to-end account of when a curated community corpus improves real agent tasks, and where it does not.

Limitations

Breadth-over-depth evaluation design. The main grid prioritises breadth across twelve

(harness, backbone, benchmark) combinations plus a frontier robustness check. Depth is uneven by design. All three benchmarks are run with three repeated evaluations per cell, and we report task-clustered standard errors over the per-task paired deltas (§4.2). On the high-baseline LLM-judged benchmarks (GDPVal, QwenClawBench) per-task judge noise exceeds the effect size, so the positive effect is established at the pooled benchmark level rather than cell by cell. SkillsBench additionally admits paired per-task statistics (exact McNemar on a representative run) and the logistic safety-facet regression (Appendix A.2). A larger number of repetitions and seeds per cell would tighten the per-cell estimates that the pooled analysis currently leans on.

Text-based skill scoring. The 3-facet quality framework and 19-flag taxonomy are evaluated by an LLM judge against the `SKILL.md` text content rather than by sandboxed execution. This is a deliberate scale trade-off. Judge-time text scoring is feasible at the $\sim 821,000$ -file crawl scale, whereas runtime sandboxing would require per-skill compatible execution environments not generally available for community-contributed code. Safety hard-gates and licence filtering at Stage 5 catch the most severe content-level risks at ingest time. Complementary runtime verification at retrieval-time or agent-execution-time is a principled extension for future work. A second consequence of the scale trade-off is that the quality, safety, and classification labels driving the release decision come from a single LLM judge and are not validated against human gold annotations or cross-checked against an alternative judge model. Judge bias therefore propagates into the released active set. The facet-separation analysis is an internal-consistency check, not a construct-validity study, and human-annotation and judge-robustness studies are left to future work.

Benchmark and harness coverage. The evaluation spans three publicly available benchmarks (SkillsBench, GDPVal, QwenClawBench) chosen for non-saturation on frontier LLMs and for complementary task-type coverage (deterministic-verifier software tasks, real-world knowledge work, and a real-user-distribution agent benchmark), and two `SKILL.md`-conformant agent harnesses (OpenClaw, Raven) covering the most widely adopted skill-loading convention in the public ecosystem. Per-benchmark and per-harness heterogeneity within this coverage is reported transparently in the main results (§4.6) rather than relegated to this section. Extending the evaluation to additional task types (e.g., long-horizon embodied tasks, non-English workflows) and to harnesses with substantially different tool-use interfaces or planning architectures would broaden external validity. The corpus is also English-dominant, both in its source repositories and in our evaluation, which bounds claims about non-English skills. The corpus-side findings (Stage 5 active-vs-filtered separation, class-adaptive retrieval, and the suggestive safety-facet signal) are expected to transfer modulo such adapter work, while per-cell Δ magnitudes would re-anchor against each new harness’s own no-skill baseline.

Snapshot semantics. The released active set is a snapshot of the public `SKILL.md` ecosystem at crawl time, while the community itself continues to evolve (new repositories, licence updates, shifting class distributions). We do not study temporal churn or re-curation cycles in this work, but the pipeline (per-stage reject logs, deterministic Stage 5 cut, deduplication keyed on content hash and embedding cosine) is constructed to support periodic re-runs against future ecosystem snapshots. The active set released with this paper should be read as a 2026-Q2 snapshot rather than a one-shot artifact.

Ethical Statement

Data provenance and licensing. SkillCorpus aggregates publicly available `SKILL.md` files from GitHub and community marketplaces that link to GitHub-hosted sources. Each released skill carries metadata including its source URL and content hash. We do not modify body content or claim authorship. Every released skill carries an OSI-approved permissive licence inferred from the source repository’s GitHub `spdx_id` and enforced as a Stage-5 ingest filter (Table 4, per-reason exclusion counts in Table 5). Downstream consumers redistributing the corpus or its derivatives are advised to perform their own due-diligence audit of `scripts/` content and transitive dependencies.

Safety filtering and dual-use. The Stage-5 safety hard-gate (§3.3) excludes the most severe categories from the active release set, and the soft-flag categories are recorded in each skill’s metadata for downstream per-deployment filtering. We acknowledge the dual-use risk that a publicly searchable skill corpus could be queried for adversarial-style content patterns, a residual risk that the hard-gate and released soft-flag metadata reduce but do not eliminate.

Benchmark contamination. SkillCorpus is community-contributed and pre-dates our benchmark selection (§4.1). By design the corpus aims for broad coverage of the skill ecosystem, so released skills overlap the domains of our evaluation tasks. These are general procedural knowledge rather than task-specific solutions, and the per-task gate injects at most a few topically relevant skills. We do not perform task-specific leakage audits beyond the corpus-level safety and licence filters described above, and consumers using verifier-backed benchmarks should re-audit for task-specific contamination when their evaluation regime is sensitive to it.

References

- Anthropic. 2025a. Anthropic Skill Creator: A Rubric for Agent Skill Quality. [Anthropic.com/blog](https://anthropic.com/blog), 2025.
- Anthropic. 2025b. Claude Code: An Agentic Coding Tool. [Github.com/anthropics/claude-code](https://github.com/anthropics/claude-code), 2025.
- Chen, C.; Yu, Q.; Gu, Y.; Huang, Z.; Li, H.; Liu, H.; Liu, S.; Liu, J.; Peng, D.; Wang, J.; Yan, Z.; Meng, F.; Qin, E.; Che, C.; and Hu, M. 2026. The Scaling Laws of Skills in LLM Agent Systems. *arXiv preprint arXiv:2605.16508*.
- Cho, H.; Kang, R.; and Kim, Y. 2026. SkillRet: A Large-Scale Benchmark for Skill Retrieval in LLM Agents. *arXiv preprint arXiv:2605.05726*.

- Hou, X.; Zhao, Y.; Wang, S.; and Wang, H. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Transactions on Software Engineering and Methodology*. ArXiv:2503.23278.
- Lee, K.; Ippolito, D.; Nystrom, A.; Zhang, C.; Eck, D.; Callison-Burch, C.; and Carlini, N. 2022. Deduplicating Training Data Makes Language Models Better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. ArXiv:2107.06499.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2005.11401.
- Li, F.; Tagkopoulou, P.; and Tagkopoulou, I. 2025. SkillFlow: Scalable and Efficient Agent Skill Retrieval System. *arXiv preprint arXiv:2504.06188*.
- Li, H.; Mu, C.; Chen, J.; Ren, S.; Cui, Z.; Zhang, Y.; Bai, L.; and Hu, S. 2026a. Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale. *arXiv preprint arXiv:2603.02176*.
- Li, X.; Chen, W.; Liu, Y.; Zheng, S.; Chen, X.; He, Y.; Li, Y.; You, B.; Shen, H.; Sun, J.; et al. 2026b. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670*.
- Liang, Y.; Zhong, R.; Xu, H.; Jiang, C.; Zhong, Y.; Fang, R.; Gu, J.-C.; Deng, S.; Yao, Y.; Wang, M.; Qiao, S.; Xu, X.; Wu, T.; Wang, K.; Liu, Y.; et al. 2026. SkillNet: Create, Evaluate, and Connect AI Skills. *arXiv preprint arXiv:2603.04448*.
- Lin, M.; Lu, H.; Shi, Z.; He, B.; Mao, R.; Zhang, Z.; Wu, Z.; Tang, X.; Liu, H.; Dai, Z.; Zhang, X.; Wang, S.; Dumoulin, B.; and Pei, J. 2026. Position: Agentic Evolution is the Path to Evolving LLMs. *arXiv preprint arXiv:2602.00359*.
- Lin, Z.; Ruan, B.; Liu, J.; and Zhao, W. 2025. A Large-Scale Evolvable Dataset for Model Context Protocol Ecosystem and Security Analysis. *arXiv preprint arXiv:2506.23474*.
- Ling, G.; Zhong, S.; and Huang, R. 2026. Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality. *arXiv preprint arXiv:2602.08004*.
- Liu, D.; Li, Z.; Du, H.; Wu, X.; Gui, S.; Kuang, Y.; and Sun, L. 2026a. Graph of Skills: Dependency-Aware Structural Retrieval for Massive Agent Skills. *arXiv preprint arXiv:2604.05333*.
- Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; and Chang, S. 2026b. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings. *arXiv preprint arXiv:2604.04323*.
- Meng, X.; Wang, S.; and Fang, Y. 2026. SkillRAE: Agent Skill-Based Context Compilation for Retrieval-Augmented Execution. *arXiv preprint arXiv:2605.10114*.
- Merrill, M. A.; Shaw, A. G.; Carlini, N.; Li, B.; Raj, H.; Bercovich, I.; Shi, L.; Shin, J. Y.; Walshe, T.; Buchanan, E. K.; et al. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. *arXiv preprint arXiv:2601.11868*.
- OpenAI. 2025a. Codex CLI: Lightweight Coding Agent that Runs in Your Terminal. [Github.com/openai/codex](https://github.com/openai/codex), 2025.
- OpenAI. 2025b. GDPval: Measuring LLM Performance on Real-World Economic Tasks. [Openai.com/research](https://openai.com/research), 2025.
- Park, J. S.; O’Brien, J. C.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *UIST*.
- Patil, S. G.; Zhang, T.; Wang, X.; and Gonzalez, J. E. 2023. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334*.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *ICLR*.
- Qwen Team, Alibaba Group. 2026. QwenClawBench: A Real-User-Distribution Agent Benchmark for Open-Claw. Open-source benchmark, github.com/SKYLENAGE-AI/QwenClawBench, released April 2026.
- Raven Contributors. 2026. Raven: An Open-Source Agent Framework. Open-source repository; supports SKILL.md skill-loading convention.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *NeurIPS*.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS*.
- Steinberger, P. 2025. OpenClaw: An Open-Source Autonomous Agent Framework. Originally published Nov 2025 as Clawdbot; renamed OpenClaw 2026. Skill-loading via SKILL.md convention.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*.
- Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2306.05685.
- Zheng, Y.; Zhang, Z.; Ma, C.; Yu, Y.; Zhu, J.; Wu, Y.; Xu, T.; Dong, B.; Zhu, H.; Huang, R.; and Yu, G. 2026. SkillRouter: Skill Routing for LLM Agents at Scale. *arXiv preprint arXiv:2603.22455*.
- Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; Zhang, S.; Ghosh, G.; Lewis, M.; Zettlemoyer, L.; and Levy, O. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2305.11206.

A Corpus Construction Details

A.1 Quality Flag Vocabulary

The 19-flag taxonomy used by the LLM quality judge is structured by its primary semantic dimension. Flags marked [†] are hard-gate flags whose firing forces the per-skill quality score to zero and the skill to be excluded from the released active set.

- Utility-bound (2): marketing_only, vague_purpose.
- Robustness-bound (6): placeholder, no_steps, deprecated_api, fabricated_call, syntax_error, inconsistent_doc_code.
- Safety-bound (11): destructive_no_confirm, secret_leak, network_exfil, prompt_injection[†], cmd_injection[†], unsafe_exec[†], auth_bypass[†], fact_poisoning, tos_violation, csam_risk[†], bias_content.

Flag classification follows the LLM judge prompt. The same flag may in principle constrain more than one facet (e.g., a placeholder skill primarily caps the robustness facet but may also indicate poor utility scope), but for scoring we use the single-facet assignment shown above.

Hard-gate vs soft-signal rationale. The five hard-gate flags (prompt_injection, cmd_injection, unsafe_exec, auth_bypass, csam_risk) share two properties that distinguish them from the 14 soft signals: (i) a single firing is sufficient to make the skill unsafe to ship even in conjunction with downstream sandboxing or partial-load policies, and (ii) remediation cannot be deferred to per-deployment filtering without re-vetting the body content. Soft signals such as secret_leak or destructive_no_confirm, by contrast, can be mitigated by deployment-side wrappers (credential rotation, confirmation prompts) or by skipping affected skills in a particular deployment without invalidating the broader artifact. The distinction is therefore an integrability boundary, not an absolute-severity ranking.

A.2 Quality-Score Distribution and Facet Separation

Whether the 3-facet prompt produces genuinely separate scores is subtler than Pearson r alone can answer, because real skills share latent craftsmanship that elevates all facets simultaneously, so some positive correlation is expected even under independent measurement. For each facet pair we report both Pearson r and the off-diagonal mass $\Pr(a \geq 8 \wedge b \leq 4)$ against the product-of-marginals independence baseline, computed on the 96,401-skill active set and visualised in Figure 6. Observed off-diagonal mass sits below the independence baseline for all three pairs. Among the

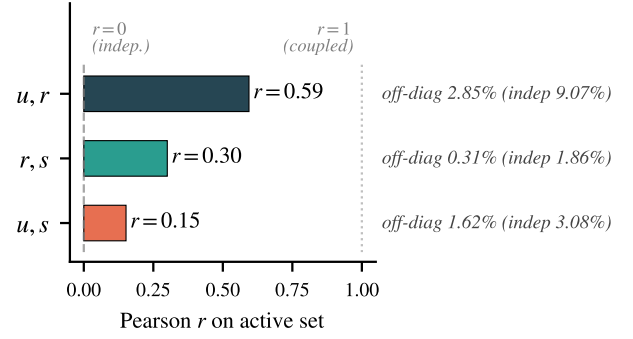


Figure 6: Three-facet independence on the active set ($N=96,401$). Bar length is the Pearson r for each facet pair (dashed line at $r=0$ marks statistical independence). Italic annotations report off-diagonal mass $\Pr(a \geq 8 \wedge b \leq 4)$ versus the independence baseline.

craftsmanship-coupled pair, the prompt still isolates a 2.85% “description-good / body-weak” quadrant ($u \geq 8 \wedge r \leq 4$) that a one-dimensional composite score cannot represent. Because the subscores are integer 0–10 values subject to soft-flag capping, the Pearson r is an attenuated estimate of association.

Per-task outcome relationship. The composite score does not predict per-task pass rate on any benchmark (all $|r| < 0.10$, $p > 0.35$). Only the safety facet carries a suggestive per-task signal, a pooled SkillsBench logistic giving $\hat{\beta}_{\text{safety}} = +1.03$ ($z = +2.29$, $p=0.022$) that rests on a single grid and would not survive a multiple-comparison correction, so whether it can serve as a ranking cue beyond topical relevance merits validation on further grids.

Score distribution. On the 96,401 active skills the composite quality score (on the $[0, 1]$ scale) has mean 0.690. The bucket-level distribution is in Table 3.

Quality bucket	% of active
$q < 0.3$	2.8
$q \in [0.3, 0.5)$	5.5
$q \in [0.5, 0.7)$	32.9
$q \geq 0.7$ (curated)	58.8

Table 3: Quality-bucket distribution on the active set.

A.3 Source Prior

Each source s in the corpus is assigned a prior

$$\begin{aligned} \text{prior}(s) = & 0.95 \cdot \text{track}(s) \\ & + 0.05 \cdot \text{lic_rate}(s) \\ & + 0.10 \cdot \mathbb{I}[s \in \mathcal{T}], \end{aligned}$$

clipped to $[0.30, 1.00]$, where:

- $\text{track}(s) = \frac{n\bar{q}_s + K\mu_0}{n + K}$ is the Bayesian-shrunk source quality, with n the number of skills from s , $\bar{q}_s \in [0, 1]$

the source’s empirical normalised content-quality mean (§3.2), $K = 10$ a shrinkage strength in skill-count units, and $\mu_0 = 0.685$ the dataset-wide normalised content-quality mean.

- $\text{lic_rate}(s) \in [0, 1]$ is the fraction of s ’s skills carrying an OSI-permissive licence.
- $\mathcal{T}$ is a small explicit trust list (`anthropics`, `antigravity`, `karanb192`). The indicator contributes at most $+0.10$ to dampen the influence of hand-curation relative to the data-driven track score.

$K = 10$ was chosen *a priori* from the heuristic "ten observations is the minimum credible sample". We did not tune it on downstream task performance.

A.4 False-Positive Audit of Safety Regex Patterns

The Stage 5 safety hard-gate applies a single pre-judge malware regex (`blocked.malware`). A set of additional regex patterns (e.g., suspicious-string heuristics) are retained for audit logging only. A stratified 100-skill manual audit indicates that several audit-only regex patterns have empirical false-positive rates above 90%, which is why they are not used as filtering gates. Detailed false-positive counts per regex pattern are provided in the released ingest report.

A.5 Licence Breakdown and Exclusion Counts

All 96,401 released skills carry an OSI-approved permissive licence. Table 4 reports the licence composition of the active set, dominated by MIT (88.3%) and Apache-2.0 (10.5%). Table 5 itemises the 3,795 skills excluded at the Stage-5 licence filter by exclusion reason. Missing `LICENSE` files and unreachable source repositories account for $\sim 87\%$ of exclusions, while explicitly incompatible terms (copyleft or non-commercial/share-alike) account for under 9%.

Licence bucket (all GREEN)	%
MIT / MIT-0	88.3
Apache-2.0	10.5
Other OSI-permissive [†]	1.2
Total	100.0

Table 4: Licence breakdown of the released active set ($N=96,401$). [†]CC0 / BSD / MPL-2.0 / Unlicense / ISC / Mulan-PSL-2.0 / WTFPL / 0BSD.

Exclusion reason	count
Source repo has no <code>LICENSE</code> file	1,756
Source repo unreachable via GitHub API	1,563
Copyleft (GPL / AGPL / LGPL)	228
Non-commercial or share-alike (CC-NC/SA/ND)	86
Custom or unparsed licence string	162
Total excluded	3,795

Table 5: Skills excluded at the Stage 5 licence filter to guarantee commercial redistributability.

A.6 Crawl Source Inventory

The SkillCorpus crawl (§3.1) is configured by a single machine-readable registry of 62 sources. Each entry declares one of five ingestion mechanisms, and all mechanisms feed a shared discovery entry point, so adding a source is a one-line registry change. The mechanisms are itemised below with per-mechanism counts and representative sources.

Mechanism (sources)	Representative sources
Direct repository clones (38)	<code>anthropics/skills</code> , <code>openclaw/skills</code> , <code>openclaw/clawhub</code> , <code>antigravity</code> , and the long tail of repositories tagged <code>claude-skills</code> or <code>agent-skills</code> .
Awesome-list README scrapes (19)	<code>VoltAgent/awesome-agent-skills</code> , <code>karanb192/awesome-claude-skills</code> , <code>Chat2AnyLLM/awesome-claude-skills</code> , and others.
Marketplace index APIs (2)	<code>SkillsMP</code> , <code>SkillsDirectory</code> .
Sitemap crawl (1)	<code>Skills.sh</code> .
JSON catalogs (2)	<code>LobeHub index</code> , <code>SkillManager</code> .

Table 6: Crawl sources by ingestion mechanism. The complete registry ships with the released artifact as a machine-readable file.

B Retrieval and Evaluation Details

B.1 Standalone Retrieval Performance

We evaluate the recall-and-rerank stack as a standalone retrieval system on three candidate pools using the SkillRouter evaluation protocol (Zheng et al. 2026): the SkillRouter Easy tier (standard large-pool retrieval), the SkillRouter Hard tier (with adversarial distractors), and our own SkillCorpus pool (with the ground-truth SkillsBench skills explicitly injected at evaluation time but excluded from training and from the downstream agent). We compare against two baselines: `BASE` (off-the-shelf Qwen3-Emb-0.6B and Qwen3-Rank-0.6B) and `SKILLROUTER` (Zheng et al. 2026) using its released models. Performance is measured with Hit@1 and Recall@10 on the 75 core SkillsBench tasks (excluding 12 generic-only cases per the SkillRouter protocol).

Method	Metric	Easy	Hard	SkillCorpus
BASE	Hit@1	0.627	0.613	0.400
	Recall@10	0.600	0.599	0.635
SKILLROUTER	Hit@1	0.760	0.720	0.613
	Recall@10	0.729	0.696	0.691
OURS	Hit@1	0.760	0.760	0.720
	Recall@10	0.722	0.679	0.718

Table 7: Standalone retrieval performance (Hit@1 and Recall@10 on the 75-task core SkillsBench evaluation set).

The Easy / Hard performance shows cross-pool generalisability, since our stack is competitive on tiers it was not

trained on. The SkillCorpus performance reflects the expected advantage of train-time and evaluation-time distributions matching. The gap between Easy and SkillCorpus for both SKILLROUTER and OURS (-14.7 and -4.0 pp on Hit@1 respectively) indicates that SkillCorpus is intrinsically harder than the Easy tier, consistent with its larger size and more diverse distractor mass.

B.2 Cross-Benchmark Domain Coverage

Figure 7 documents the breadth of task domains the corpus is exercised against. It plots the 407 evaluated tasks over 26 domain labels in each benchmark’s native taxonomy (8 SkillsBench categories plus a residual Other bin, 9 GDPVal sectors, and 8 QwenClawBench categories), with bar length giving the task count and bar colour the per-domain Δ on the strongest cell (Raven $\times$ Q-397B). Those per-domain Δ values are coarse and noisy at $N \leq 25$ per cell, so they mark only where coverage is dense or thin and are not per-domain effect claims, which the main text makes at the pooled level (§4.2).

At the task level, the relevance of the best-retrieved skill predicts the gain more directly than these coarse domain bins. Beyond the binned summary in the main text (Figure 5), Figure 8 plots each SkillsBench task’s Δ against the top reranker score of its best-matched skill, a proxy that reflects both whether the corpus holds a matching skill and whether retrieval surfaces it. The positive slope holds across all four cells (Pearson $r=0.31$ – 0.40 , all $|t|>2.9$, $n=83$), the panel showing the strongest cell. The score is essentially uncorrelated with the no-skill baseline ($|r| < 0.05$), so partialling out task difficulty leaves the association unchanged (partial $r=0.34$ – 0.40).

B.3 Class-Benchmark Alignment

Table 8 reports the per-benchmark class over-selection underlying the class-adaptive selection result of §4.6. For each benchmark it lists the skill classes most over-represented in the post-gate selected pool relative to their corpus-wide frequency, showing that each concentrates the gate on a different, task-appropriate set of classes. All three reject the null of the same class distribution as the corpus (Pearson χ^2 goodness-of-fit, $p < 10^{-10}$).

Benchmark	Over-selected classes (post-gate % vs corpus %)
SkillsBench	DATA (38 vs 14), DOC-PROC (13 vs 3), SECURITY (6 vs 4)
GDPVal	WRITING (31 vs 8), DATA (29 vs 15), WORKFLOW (14 vs 5)
QwenClawBench	DEVOPS-INFRA (13 vs 8), META (11 vs 4), TESTING (10 vs 6)

Table 8: Per-benchmark 16-class over-selection relative to the corpus-wide baseline.

B.4 Case Studies: a Rescue and a Break

We illustrate the end-to-end pipeline with two worked examples, one task that skills rescue and one of the rare

tasks that skills break. The rescue is manufacturing-codebook-normalization, a SkillsBench industrial-physical-systems task that the frontier check of §4.3 (Claude Opus 4.7 on the OpenClaw harness) fails without skills and passes with them.

Task. Normalise free-form defect-reason texts from manufacturing test logs (typos, abbreviations, mixed-language fragments) into a standardised codebook: segment each raw text, assign a valid code per segment subject to per-station code validity, and emit calibrated confidence scores that distinguish UNKNOWN from low-evidence known predictions.

Retrieved skills. The gate (§3.4) selects two skills from the corpus: a domain skill, manufacturing-failure-reason-codebook-normalization, and a generic utility, fuzzy-match. The domain skill’s body specifies the procedure rather than the topic:

“If `entry.stations` is not `None`, the predicted code should only be considered valid when the record station matches [...] otherwise the code should be rejected. [...] consider multiple evidence sources [...] (overlap or fuzzy similarity between `span_text` and codebook text [...], station compatibility, `fail_code` alignment, `test_item` alignment, and conflict cues), and normalise the score to a stable range [0.0, 1.0].”

Without skills. The agent writes a plausible solution built on a hard-coded phrase dictionary with station filtering, but no multi-evidence scoring and no confidence calibration. It passes 15 of 16 verifier tests and fails the semantic-alignment check. Only 66.2% of its non-UNKNOWN predictions carry non-trivial lexical evidence against the codebook (the verifier requires $\geq 70\%$), i.e. it confidently emits codes the text does not support. Binary task reward: 0.

With skills. The agent implements the skill’s procedure directly: multi-evidence weighted scoring, station-scope penalties, a deterministic tie-break for near-tied candidates, and calibrated confidence with an explicit UNKNOWN band. All 16 tests pass, with 19% less wall-clock time and 20% fewer total tokens than the failing no-skill run.

The break: exceltable-in-ppt. The complementary failure mode appears on one of the two tasks the strongest open cell (Raven $\times$ Q-397B) breaks (no-skill passes, with-skill fails). The task is to update an exchange rate inside an Excel table embedded in a PowerPoint slide, preserving its formulas. Without skills the agent improvises a working path through the four-layer nesting, from the PPTX part through the OLE blob and its embedded ZIP down to the sheet XML, patches the cell, and re-packs the blob (8/8 verifier tests pass). With skills, the gate injects two topically relevant Office skills whose recipes all operate on top-level files (`load_workbook("file.xlsx")`, `Presentation("file.pptx")`). The agent follows the recipes, hits an API wall (property ‘blob’ of ‘_OleFormat’ has no setter), and never replaces the embedded object. The verifier reads `nan` where the table should be (6/8). The skills are on-topic, but their procedures do not extend to this task’s structure, and having a recipe displaced the improvisation that succeeds without one.

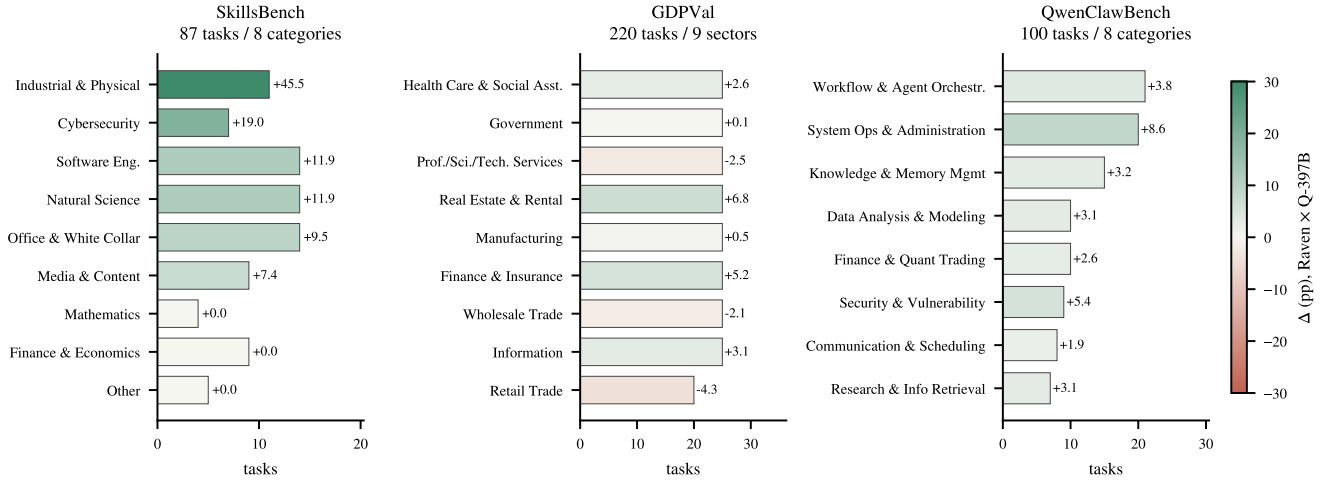


Figure 7: Cross-benchmark domain coverage of the evaluation, each panel in its benchmark’s native taxonomy.

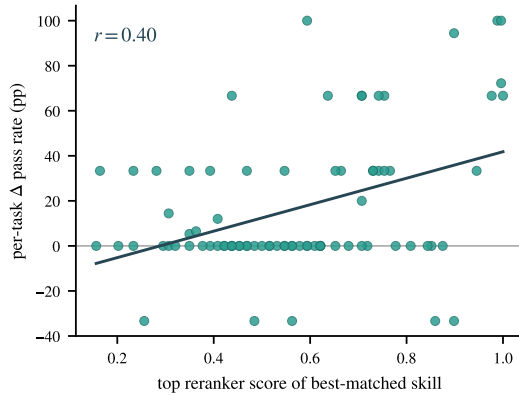


Figure 8: Per-task SkillsBench gain versus corpus coverage (top reranker score of the best-matched skill), Raven $\times$ Q-397B ($n=83$, $r=0.40$).

Takeaway. The rescue and the break are two sides of the same property. Skills transfer procedures, not topics. When the injected procedure matches the task’s structure, as with the multi-evidence scoring formulas and tie-breaks above, it supplies exactly what the model does not reliably improvise. When it does not, as with flat-file recipes against an embedded object, following it can displace an improvisation that would have succeeded. This is also one concrete reason composite quality scores cannot predict per-task outcomes (§4.6). The skills in both cases are well-crafted, and what differs is procedure–task fit.

C Compute and Reproducibility

C.1 Per-Stage Compute

The ingest pipeline was served on an internal GPU cluster using Qwen3.5-397B-A17B-GPTQ-Int4 as the LLM judge for quality scoring, classification, and embedding-based dedup.

This judge is an open-weight model, publicly available on Hugging Face and via OpenRouter, so the quality, safety, and classification labels can be re-derived independently. The internal cluster refers only to where we ran it. The compute reported below is the cumulative total over the crawl that produces the released active set.

Ingest compute. The ingest pipeline issued approximately 269,000 LLM-judge calls in total: 101,111 per-skill quality + 3-facet score calls on the post-dedup survivors (one call per row of the released `quality_judgments` table), 101,111 single-label 16-class classification calls on the same set, and 66,751 pair-wise dedup adjudication calls on the borderline embedding-cosine near-duplicate pairs (§3.1), per the released dedup report. Calls were issued at standard worker concurrency on an internal GPU cluster running Qwen3.5-397B-A17B-GPTQ-Int4. The LLM-judge stage dominates wall time, while content-hash, length-gate, and licence-API stages run at file-IO / network speed and contribute negligible compute.

Evaluation compute. The main grid comprises 24 configurations (2 conditions $\times$ 4 cells $\times$ 3 benchmarks), each run three times, on the two open Q-27B and Q-397B backbones, served through Volcano (and, for one cell of OpenClaw, DashScope). The Opus 4.7 frontier robustness check (2 conditions $\times$ 87 SkillsBench tasks = 174 task runs) consumed approximately 24 million API tokens in total, estimated from the per-task token counters recorded in the released eval cache (mean 158K total tokens per task for the SkillCorpus condition and 122K for the no-skill baseline, 87 tasks each). The per-task safety-facet regression on SkillsBench (§4.6) reuses traces from the main grid and adds negligible incremental cost. Aggregate wall-clock and GPU-hour accounting per ingest pass and per evaluation cell is provided in the released compute log alongside the corpus artifact.

C.2 Reproducibility

The corpus artifact (SQLite database, embeddings, quality cache, per-stage reject reports) and the ingest pipeline source will be released upon acceptance, linked from a public Hugging Face dataset entry and a GitHub repository hosting the pipeline code under a permissive licence.

Multi-run merge protocol. The released active set is the cumulative union of multiple ingest passes over the same SQLite database. Content-hash and name-hash dedup operate across all passes, so ingesting a repository whose skills are content-identical to rows already in the database produces no duplicate rows. The released active set is the database snapshot at release time restricted to rows with `active = 1` and `deleted = 0`, the canonical release predicate. Per-pass reject counts and the chronological sequence of pipeline configuration deltas are preserved in the released ingest report.

Reproducing the release predicate. A consumer reproducing the released set from the SQLite database applies the predicate `active = 1  $\wedge$  deleted = 0` to the `skills` table, which yields exactly 96,401 rows. All downstream tables (`quality_judgments`, `source_priors`, `vec_skills`) are keyed on `content_hash` or `skill_id` and join on the released set.